CSV, JSON y XML : lo mismo guardado de 3 formas

Asier García Catalán · 2º DAM · Acceso a Datos · 26/10/2026

1. Mi coche (el 700) en los tres formatos

CSV

```
Id;Marca;Modelo;
Anio;Combustible;
Color;Kilometros;
Precio
700;Kia;Ceed;
2014;Gasolina;
Negro;108400;
6290
```

JSON

```
{
  "Id": 700,
  "Marca": "Kia",
  "Modelo": "Ceed",
  "Anio": 2014,
  "Combustible": "Gasolina",
  "Color": "Negro",
  "Kilometros": 108400,
  "Precio": 6290
}
```

XML

```
<Coche>
  <Id>700</Id>
  <Marca>Kia</Marca>
  <Modelo>Ceed</Modelo>
  <Anio>2014</Anio>
  <Combustible>Gasolina</Combustible>
  <Color>Negro</Color>
  <Kilometros>108400</Kilometros>
  <Precio>6290</Precio>
</Coche>
```

↳ Los datos son los mismos. Lo que cambia: en el CSV los nombres salen una vez, en el JSON en cada dato y en el XML dos veces (abrir y cerrar).

2. Las partes de cada uno

CSV

Id;Marca;...
1;Renault;...
1ª línea = **cabecera**
cada línea = **registro**
el ; = **separador**

JSON

```
[ {
  "Marca": "Kia",
  "Id": 700,
  "Nuevo": false,
  "Matricula": null
} ]
```

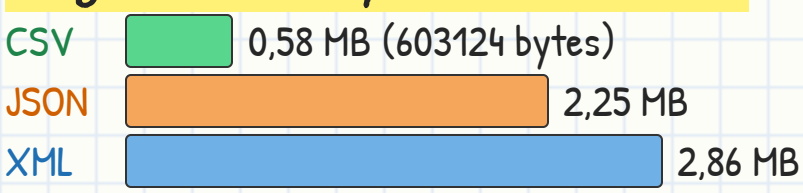
[ ] **array** (mi List) · { } **objeto**
"clave": valor → texto, número, true/false, null
(Nuevo y Matricula son de ejemplo, mi clase Coche no los tiene)

XML

```
<?xml version="1.0"?>
<ArrayOfCoche>
  <Coche>
    <Marca>Kia</Marca>
  </Coche>
</ArrayOfCoche>
```

declaración, **raíz** y **elementos**
<Marca> abre · </Marca> cierra

3. ¿Cuánto ocupan los 12.000?



El JSON ocupa **3,9 veces** más que el CSV y el XML casi 5 (**4,98**).
En el XML el **83,5%** son etiquetas y espacios, no datos.
¿Por qué? Porque el JSON y el XML repiten los nombres de los campos en los 12.000 coches, y el CSV solo los pone una vez.

4. ¿Y si los rompo?

- CSV con un ; de más → **FormatException**
- CSV sin color → no falla, pero 1 sospechoso
- JSON sin una coma → **JsonException**
- JSON con "precio" → no falla, precio 0
- XML sin </Color> → **InvalidOperationException**
- XML con <marca> → no falla, sin marca

Lo que he sacado: los errores que saltan son fáciles de ver. Los que no fallan son los peores, porque si no miras los datos ni te enteras. Cómo evitarlos:
· CSV: usar **int.TryParse** y mirar que haya 8 campos
· JSON: **PropertyNameCaseInsensitive = true**
· XML: validarlo con un esquema (XSD)
· Siempre: contar los coches con campos vacíos o a 0

5. Dónde los he visto

- CSV: `prestamos_biblioteca.txt` (Tema 3). Es una tabla enorme y así ocupa poco.
- JSON: `package.json` en Angular. Tiene datos dentro de datos y es lo que usa JavaScript.
- XML: `MedidorFormatos.csproj`. Visual Studio lo usa para la configuración del proyecto.

Hecho con Visual Studio (C#) y Canva, con algo de ayuda de IA (Claude) para revisar · Fuentes: apuntes del Tema 2 y Microsoft Learn